RISC-V ISA

Labo Architecture des ordinateurs

Contenu

1 Objectifs . . . 1
2 Instructions . . . 2
2.1 Constantes (Immediate) . . . 3
2.2 Calculs de base . . . 3
2.3 Accès mémoire . . . 3
2.4 Algorithmes . . . 4
3 Programmation impérative . . . 5
3.1 Branching . . . 5
3.2 Loops . . . 5
3.3 Fonctions . . . 6
3.4 Algorithmes . . . 7
4 Assembler / Disassembler . . . 9
4.1 Programme Ripes . . . 9
4.2 Analyse 1: Gestion de la mémoire dans Ripes . . . 11
4.3 Analyse 2: Travail d'un compilateur . . . 13
4.4 HEIRV-32 . . . 14
4.5 Reverse Engineering . . . 16

1 Objectifs

Ce laboratoire est divisé en plusieurs blocs réalisés sur plusieurs semaines. Le but est de se familiariser avec le langage assembleur du Reduced Instruction Set Computer (RISC-V).

- La partie Chapitre 2 du laboratoire porte sur les instructions individuelles.
- La partie Chapitre 3 du laboratoire nous amène à la programmation impérative, respectivement aux boucles et aux fonctions.
- La dernière partie Chapitre 4 du laboratoire est consacrée au travail sur le compilateur : l'assemblage et le désassemblage d'un programme (passage d'un langage haut-niveau à du code machine et inversement).

2 Instructions

Dans cette première partie du laboratoire, nous allons travailler avec l'interpréteur RISC-V sur le site https://course.hevs.io/car/riscv-interpreter/, voir illustration Figure 1. Celui-ci permet aux registres d'écrire en mémoire et d'exécuter le code étape par étape. Ces outils en ligne vous aideront à résoudre et à contrôler les tâches.

> Seules les instructions listées sur cette page sous **Supported Instructions** peuvent être utilisées. Les pseudo-instructions ne sont pas prises en charge.

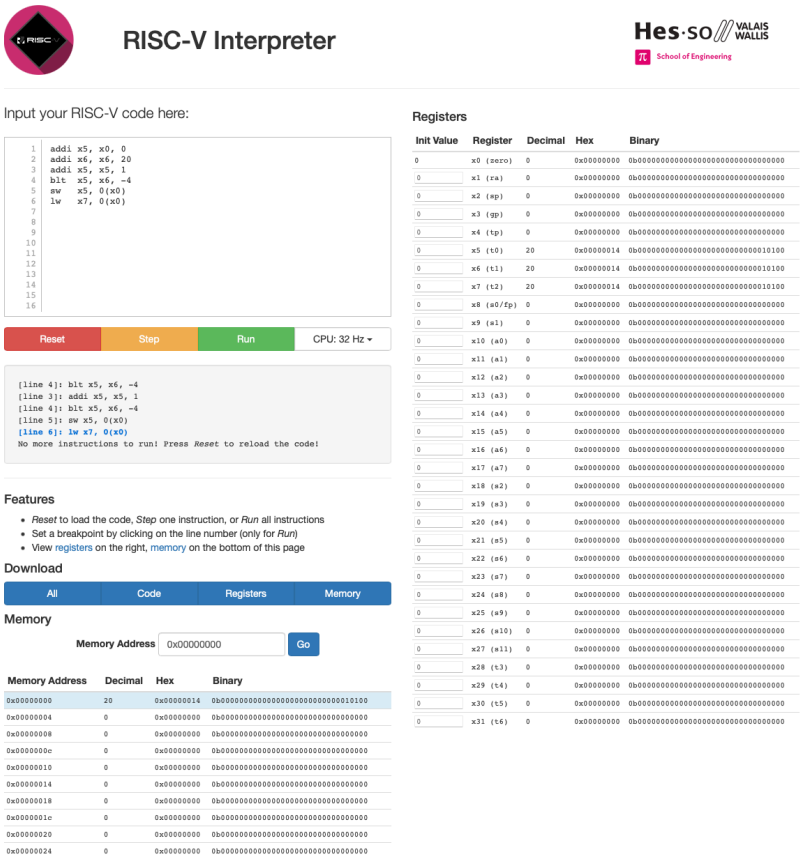

Figure 1 - Interpréteur RISC-V en ligne

> Attention aux types des variables !
> - Le type **int** est considéré de taille 32 bits signé.
> - Le type **unsigned int** est considéré de taille 32 bits non-signé.
> - Si il est suivi d'un nombre (ex: **int16_t**), cela signifie que la variable est sur x bits (ici 16). Si précédé d'un **u**, il est non-signé.
>
> **uint8_t** est donc un byte non-signé, tandis que **int8_t** est un byte signé.

2.1 Constantes (Immediate)

Écrire le code assembleur RV32i pour les instructions suivantes :

a)

```
int a = 10;
int b = 0;
a = a + 4;
b = a - 12;

int i = 0;
int x = 2032;
int y = -78;
```

b)

```
int a = 0xABCDE123;

int b = 0xFEEDA987;
```

2.2 Calculs de base

Écrire le code assembleur RV32i pour les instructions suivantes :

a)

```
int b = 1;
int c = 2;
int d = 5;
a = b + c - d;
```

b)

```
int b = -1;
int c = 2;
int d = -78;
a = b + c - d;
```

c)

```
int b = -12;
int c = 2023;
int d = 22;
a = b + c - d;
```

2.3 Accès mémoire

Écrire le code assembleur RV32i pour les instructions suivantes. Afin de tester votre solution avec des valeurs correctes, utilisez le code fournit pour initialiser quelques valeurs. Le tableau se trouve à l'adresse mémoire pointée par le registre **a0**. Contrôlez les résultats en analysant registres et mémoire après l'exécution de votre code.

```
# We have an array of int, i.e., multiple ints one after the other in memory such as
[int0][int1][int2] ...
# The notation mem[x] means getting the x th element of that array

addi a0, x0, 0x08  # Init memory pointer
addi t0, x0, 10    # Init value
sb t0, 1(a0)       # Write subpart of mem[0]
sb t0, 10(a0)      # Write subpart of mem[3]

# ...

int a = mem[0];
int b = mem[3];
mem[5] = 42;
int c = mem[5];
```

2.4 Algorithmes

Écrire le code assembleur RV32i pour les algorithmes suivants, en utilisant uniquement les instructions de base sans *loops*. *conditionals* ainsi que *branches* :

1. Dans un système, il peut être nécessaire d'attendre sans rien faire, sans modifier l'état actuel du système (pas d'accès mémoire, pas de modification des registres). Cette opération est communément appelée **NOP** (**NO Op**eration).
   1. Proposez une instruction pour ce faire.
   2. Testez qu'aucun registre ni valeur mémoire n'est effectivement modifié.
2. Calculer les 10 premiers nombres de Fibonacci. Enregistrer chaque nombre dans un registre différent.

3 Programmation impérative

3.1 Branching

3.1.1 If / else

```
int a = 1, b = 2, c;

if(a == b) {
  c = 1;
}
else {
  c = 0;
}
```

3.1.2 Switch case

```
int a, b;

switch(b) {
  case 0:
    a = 17;
    break;
  default:
    a = 99;
}
```

3.2 Loops

3.2.1 While / Do While

a)

```
int a = 10;

do{a = a - 1;}
while(a != 0);
```

b)

```
int a = 10;

while(a >= 0)
{a = a - 1;}
```

3.2.2 For

a)

```
unsigned int a = 0, i;
// mem array is at address saved in
register a0
for(i = 0; i < mem[0]; i = i + 1) {
  a = a + 2;
}
```

b)

```
// An array of 10 bytes
uint8_t myArray[10] = ...
// ...

// Let say myArray[0] is at the address
saved in register s0.
// Arrays are contiguous in memory:
myArray = [el0][el1][el2]...[elN]

int i;

for(i = 0; i < 10; i = i + 1) {
  myArray[i] = myArray[i] - 5;
}
```

3.3 Fonctions

Bien qu'un programmeur seul puisse agencer son code comme il l'entend, certaines conventions ont été établies pour permettre l'interopérabilité de plusieurs sources de code entre elles.

La fonction en appelant une autre est nommée **caller** et la fonction appelée nommée **callee**. Elles doivent être synchronisées sur la façon dont sont passés les arguments, quels registres doivent être conservés …

Les notions suivantes sont les règles essentielles:

- Les registres **a0** à **a7** permettent de passer des arguments. Si plus sont nécessaires, ils sont passés sur le stack.
- Le résultat est placé dans le registre **a0**.
- Le **caller** enregistre l'addresse de retour (PC + 4) générée par l'instruction **jal** dans le registre **ra**.
- Le **callee** ne doit pas réécrire l'adresse de retour, le stack ou encore les registres sauvés sXX. Si elles sont utilisées, l'espace doit être réservé sur le stack et ces informations doivent être stockées puis restaurées.

> Le **stack pointer / sp / x2** est à la valeur 0 par défaut sur l'interpréteur RISC-V Interpreter. Vous devez manuellement l'initialiser à une valeur suffisament grande pour éviter de toucher à la mémoire réservée pour le programme. 2'000 est généralement suffisant pour les exercices suivants.

a)

```
doNothing();

...

void doNothing() {
  return;
}
```

b)

```
int a = 1, b;
b = callA(a);

...

// Functions can be
// optimized at will

int callA(int v1) {
  v1 = v1 * 2;
  return callB(v1);
}

int callB(int v1) {
  v1 = v1 + 12;
  return v1;
}
```

3.4 Algorithmes

3.4.1 Nombre au carré

Un nombre mis au carré n'est autre que la multiplication dudit nombre par lui-même. Mais ici, nous ne travaillons qu'avec le set d'instruction RV32I:

- Proposer une fonction capable de calculer $n * n$ par une boucle d'additions.
- Tester avec n = 5, 10 et 20.
- Généraliser la fonction pour pouvoir lui fournir deux paramètres en calculant $i * j$.
- Tester avec i = 5, j = 100. Tester avec i = 100, j = 5. Que constatez-vous ?
- Optimiser la fonction pour que l'ordre des opérandes n'influe pas sur le temps de calcul.

3.4.2 Fibonacci

La suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux précédents. Elle commence par `0, 1, 1, 2, 3, 5, 8, 13 ...`.

3.4.2.1 Itération

Le but est de calculer le N'ième terme de la suite de Fibonacci d'un nombre entier non-signé grâce à une boucle. Coder l'algorithme sous forme de fonction selon la recette suivante:

```
unsigned int n = 5; // Fibonacci term to calculate
unsigned int fib1 = 0; // First Fibonacci number
unsigned int fib2 = 1; // Second Fibonacci number

for (unsigned int i = 0; i < n; i++) {
    unsigned int res = fib1 + fib2; // Calculate the next Fibonacci number
    fib1 = fib2; // Update the first number
    fib2 = res; // Update the second number
}
// The loop will run n times, and at the end, fib1 will hold the nth Fibonacci number
```

3.4.2.2 Récursion

La notion de récursivité implique qu'une fonction se rappelle elle-même pour calculer un résultat, de la même manière qu'elle appellerait une autre fonction.

Le but est de calculer la suite de fibonacci d'un nombre entier non-signé selon l'algorithme suivant:

```
unsigned int fibonacci(unsigned int n){
  if(n == 0){return 0;}
  else if(n == 1) {return 1;}
  else {
    unsigned int a1 = fibonacci(n-1);
    unsigned int a2 = fibonacci(n-2);
    unsigned int a3 = a1 + a2;
    return a3;
  }
}
```

- Implémenter la fonction donnée avec RV32I
- Tester et valider la fonction
- Quelles sont les valeurs n supportées ?

Recette

```
fibonacci:
  ... # return function if n == 0

  ... # return function if n == 1

  # we will call a function, so we need to save context on stack
    # ra must be saved, else we won't be able to exit this function properly
    # n (a0) too, since we need it afterward and A registers are saved by caller
    #   => reserve 2*4 Bytes on stack pointer and save registers
  ...

  # fibonacci(n-1)
  a0 = a0 - 1
  jal ra, fibonacci

  # a0 now contains fibonacci(n-1)
    # save it on stack (you may reserve one more slot on stack)
  ...

  # calculate fibonacci(n-2)
    # since we called a function, n (a0) may have been overwritten.
    # we need to read the original N back from stack to calculate n-2
  ...
    # fibonacci(n-2)
  a0 = a0 - 2
  jal ra, fibonacci

  # Sum up the results
    # fibo(n-2) is in a0
    # fibo(n-1) is saved on stack, so read it back
  ...
  ... # calculate fibonacci(n-1) + fibonacci(n-2)
  ... # put it in a0 for return value

  # restore context
  ... # restore ra from stack
  ... # restore stack pointer: from 2 to 3 * 4 bytes, depending how you managed stack

  jalr x0, 0(ra) # finish
```

Tâche optionnelle: Mémoïsation

Il est possible d'accélérer grandement le calcul en utilisant la mémoïsation. Le principe est de réserver de la place mémoire et de stocker les résultats déjà connus pour chaque n calculé et de réutiliser cette valeur directement.

- Implémenter la fonction donnée. RV32IM est conseillé ici.
- Tester et valider la fonction
- Comparer les algorithmes

4 Assembler / Disassembler

4.1 Programme Ripes

Dans ce laboratoire, vous pouvez continuer à utiliser l'interpréteur en ligne - Figure 1. Aussi, le programme **Ripes** est également disponible, voir l'illustration Table 1.

Il supporte tout le set RV32I, les labels ainsi que les pseudo-instructions.

Vous devez d'abord le télécharger et le configurer. Ou si vous utilisez la version en ligne (https://ripes.me/), il vous suffit de la configurer, mais vous perdez la possibilité d'écrire du code C.

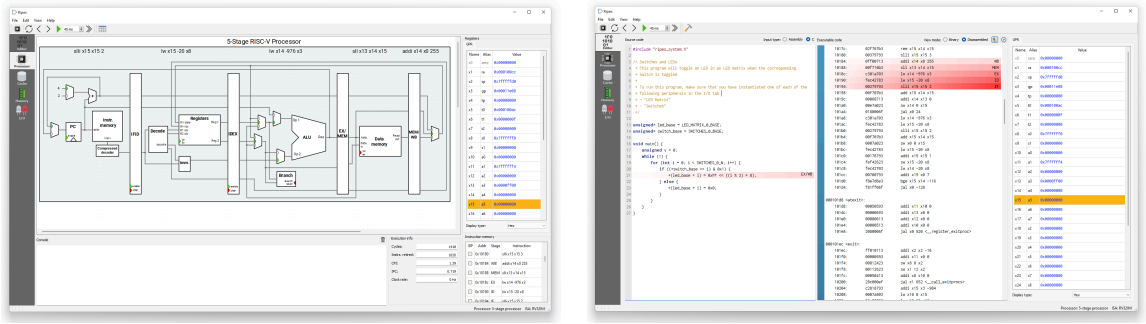

Table 1 - Interface graphique de développement de Ripes

> Sur les PCs de laboratoire, Ripes et le compilateur **gcc** énoncés ci-après sont trouvables sous **C:/eda/RiscV**.

4.1.1 Installation Ripes

1. Téléchargez la dernière version du programme pour votre plateforme en suivant le lien : https://github.com/mortbopet/Ripes/releases.
2. *(OPT) Téléchargez la version suivante de la chaîne d'outils RISC-V -GNU pour votre plateforme en suivant le lien : https://github.com/sifive/freedom-tools/releases/tag/v2020.04.0-Toolchain.Only*
3. Décompressez les deux *(OPT) et copiez le dossier RISC-V -GNU-Toolchain dans le dossier Ripes.*
4. Lancez Ripes et configurez les paramètres du compilateur dans : **Edit** → **Settings**. *(OPT)Pour cela, vous devez sélectionner le fichier* ***/riscv64-unknown-elf-gcc-8.3.0.exe***.

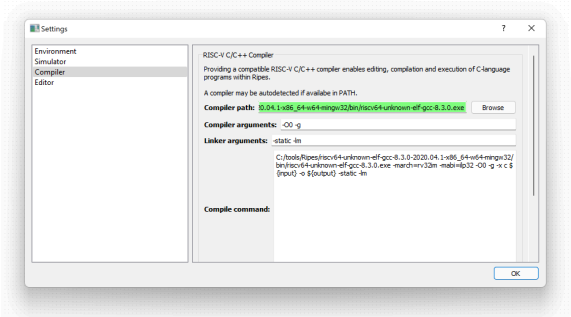

Figure 2 - Paramètres de la chaîne d'outils de Ripes

4.1.2 Setup Ripes

1. Dans les paramètres du processeur, sélectionnez le processeur monocycle RISC-V → 32-bit → sans extensions ISA.

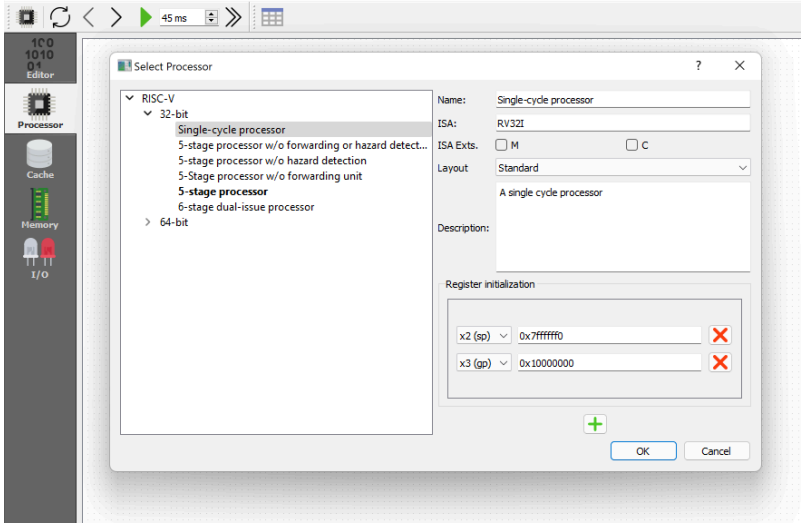

Figure 3 - Paramètres du processeur de Ripes

> Utilisez Ripes pour réaliser et tester vos laboratoire mais aussi votre série d'exercices. Il est capable de réagir comme un vrai processeur, vous montrant des comportements auxquels on peut ne pas s'attendre (ex. avec la réécriture de son propre programme en gérant mal les instructions L-S - Chapitre 4.2).

4.2 Analyse 1: Gestion de la mémoire dans Ripes

Ripes permet de sélectionner plusieurs processeurs (voir Figure 3). Malgré que le processeur single-cycle arbore deux puces de mémoire différentes pour les instructions et les données, la mémoire est considérée comme commune, de même que généralement sur une puce physique :

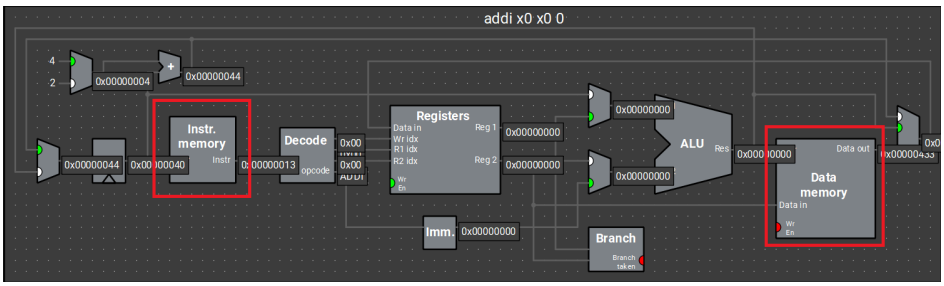

Figure 4 - Mémoires processeur single-cycle

> Cela veut dire qu'il est possible de **réécrire votre propre code par inadvertance**.

> Pour illustrer cela, exécutez le code suivant et regardez comment les instructions sont écrasées dans la mémoire d'instructions (onglet Memory).
> Utilisez le step by step.

```
# s0 counts the number of times we really loop
addi s0, zero, 0
addi s1, zero, 0 # place in memory
# t0 is the number of times we SHOULD be looping
addi t0, zero, 10

store_loop:
  sw s0, 0(s1)      # store current loop counter in memory
  addi s1, s1, 4    # increment memory place
  addi s0, s0, 1    # increment how many times we have looped
  addi t0, t0, -1   # decrement loop counter
  blt t0, zero, end
  jal zero, store_loop

end:
  nop
```

La boucle est censée enregistrer 0 à l'adresse 0, 1 à l'adresse 4, 2 à l'adresse 8 … Mais lors de la quatrième itération de la loop, l'instruction `sw s0, 0(s1)` est exécutée telle que `sw 3, 0(0x0C)`, ce qui remplace cette même instruction par 0x00000003, correspondant à une instruction `lb x0, 0(x0)`:

0x00000020	4276088943	111	240	223	254
0x0000001c	181347	99	196	2	0
0x00000018	4294083219	147	130	242	255
0x00000014	1311763	19	4	20	0
0x00000010	4490387	147	132	68	0
0x0000000c	3	3	0	0	0
0x00000008	2	2	0	0	0
0x00000004	1	1	0	0	0
0x00000000	0	0	0	0	0

Figure 5 - Réécriture mémoire

Le code a donc été modifié et ne correspond plus au but initial.

Pour éviter de tels problèmes, il faut travailler avec la mémoire au travers du **stack pointer**, comme lors d'appels de fonctions :

- Réservez de la place en mémoire en décrémentant **sp** du nombre de bytes nécessaires (pour l'exemple, nous aurions $10 * 4$ bytes)
- Travaillez avec la mémoire réservée UNIQUEMENT

4.3 Analyse 2: Travail d'un compilateur

Dans le programme Ripes Chapitre 4.1, nous avons compilé les 2 codes C suivants. Cherchez dans le code généré les commandes assembleur correspondantes. Que signifient les différentes instructions dans `main:` ?

a)

```
void main() {
  int a = 3;
}
```

```
addi sp sp -32 # stack for 8 items
sw s0 28(sp)   # saves s0 in stack[1]
as it will use it and is a callee saved
register

addi s0 sp 32  # s0 points to start of
stack
addi a5 x0 3   # a5 = 3
sw a5 -20(s0)  # stack[5] = 3
addi x0 x0 0   # nop

lw s0 28(sp)   # restore s0
addi sp sp 32  # unstack 8 items
jalr x0 x1 0   # return
```

b)

```
void main() {
  while(1) {
    int a = 3;
  }
}
```

```
addi sp sp -32 # stack for 8 items
sw s0 28(sp)   # saves s0 in stack[1]
as it will use it and is a callee saved
register

addi s0 sp 32  # s0 points to start of
stack
addi a5 x0 3   # a5 = 3
sw a5 -20(s0)  # stack[5] = 3
jal x0 -8      # jal 2 instructions
back
```

Que contiennent les deux codes assembleur ? Pourquoi le code de droite, avec plus d'instructions, est-il plus court ?

4.4 HEIRV-32

L'assembleur et désassembleur RISC-V utilisé dans ce labo peut être trouvé dans le repo sous **`isa/heirv32-asm/`** pour Windows amd64 (**`HEIRV32-ASM_1.2.3_windows_x86_64.exe`**), Linux amd64 (**`HEIRV32-ASM_1.2.3_linux_x86_64`**) et macOS arm64 (**`HEIRV32-ASM_1.2.3_macos_aarch64`**). Cet outil vous permet de transformer le code assembleur d'un processeur RV32i ou HEIRV32 en code binaire (Assembly) ainsi que de retransformer le code binaire en code assembleur (Disassembly).

- Assembly ⇨ Fichier assembleur **`*.s`** ou **`*.asm`** ⇨ Fichier binaire **`*.bin`**
- Désassemblage ⇨ Fichier binaire **`*.bin`** ou hexadécimal **`*.txt`** ⇨ Fichier assembleur **`*.asm`**.

> L'outil peut être utilisé soit en ligne de commande, soit en tant qu'interface graphique.

4.4.1 Avec l'interface graphique (GUI)

Pour cela, il suffit d'exécuter le fichier et de suivre les 3 étapes suivantes dans l'interface :

1. Choix du processeur
   - **`RV32I`** - Processeur RISC-V complet
   - **`HEIRV32`** - Processeur du projet de fin du semestre.
2. Appuyer sur le bouton **`Next`**.
3. Choix du fichier d'entrée - soit **`*.s`**, **`*.asm`**, **`*.txt`** ou **`*.bin`**. Le programme détermine automatiquement si le compilateur ou décompilateur doit être utilisé.

Ensuite, vous trouverez les fichiers générés dans le même dossier que celui sélectionné. Aussi, une fenêtre de logs reste ouverte, permettant de comprendre les étapes effectuées par le (dé)compilateur.

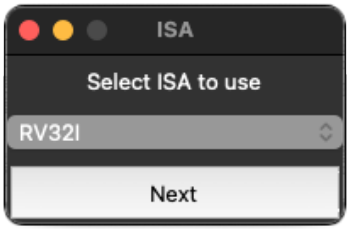

Figure 6 - GUI du programme HEIRV32-ASM

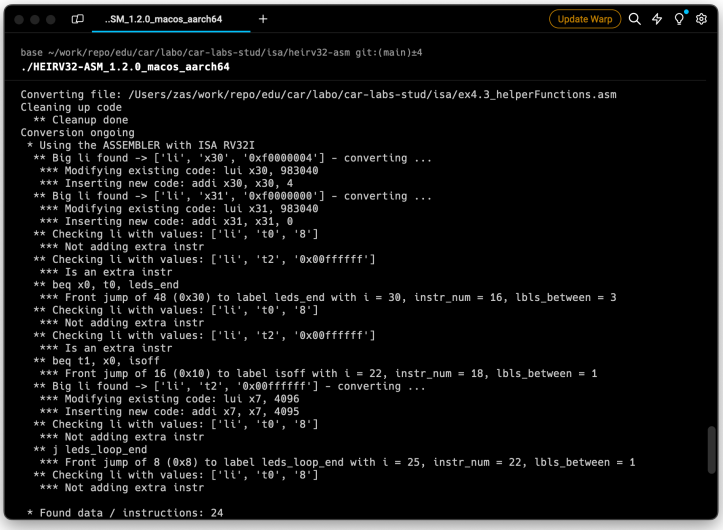

Figure 7 - Sortie du programme heirv32-asm

4.4.2 Analyse 3: (Dé)compilation

1. Enregistrez le code suivant sous un fichier **.s** ou **.asm** :

```
myLabel:
  li   x20 0b11111111111
  add  x20 x20 x20
  add  x20 x20 x20
  add  x20 x20 x20
  addi x3 x0 0
begin:
  mv   x1 x0
a:
  slt  x2 x1 x20
  beq  x2 x0 subing
  addi x1 x1 1
  beq  x0 x0 a
subing:
  addi x3 x3 -1
  beq  x3 x0 test
label_wo_beq:
  addi x30 x0 0xCC
  jal  ra begin
test:
  addi x30 x0 0b00110011
  jal  begin
l6: # This is very important
# But won't always save you
  jal  ra myLabel
```

2. Compilez-le à l'aide de HEIRV32-ASM avec les sorties **print** et **bin** activées. Pour ça, utiliser le GUI avec le set **RV32I**, ou en lignes de commandes:

   ./dist/HEIRV32-ASM_1.1.3_xxx -i RV32I -f ./my/file/disassembly-01.s -t pb

   Un fichier **.txt** est créé, qui est une version hexadécimale lisible par les humains du fichier **.bin** généré.
3. Maintenant, décompilez-le fichier **.txt** de la même manière pour recréer un fichier assembleur, soit à travers le GUI avec le set **RV32I**, soit en lignes de commande:

   ./dist/HEIRV32-ASM_1.1.3_xxx -i RV32I -f ./my/file/assembled-01.txt -t pb

   Cela crée un fichier **xxx_disassembly.s**.
4. Qu'est-ce qui change entre les deux codes ? Donnez 3 points.
5. A quoi peut servir l'instruction **jal ra myLabel** dans ce contexte ?

4.5 Reverse Engineering

> **En cas de symptômes épileptiques, ne pas continuer avec le Chapitre 4.5.**
>
> Les LEDs présentées en Table 3 vont clignoter rapidement de blanc à noir.

Vous travaillez sur un projet en binôme sur un processeur RISC-V quelque peu modifié.

En effet, pour se simplifier la vie et communiquer avec le monde extérieur, ce dernier peut lire l'état de 4 boutons reliés au processeur en lisant l'adresse mémoire **0xf0000000**, et allumer 8 leds en écrivant aux adresses mémoire **0xf0000004** (led 0) à **0xf0000020** (led 7):

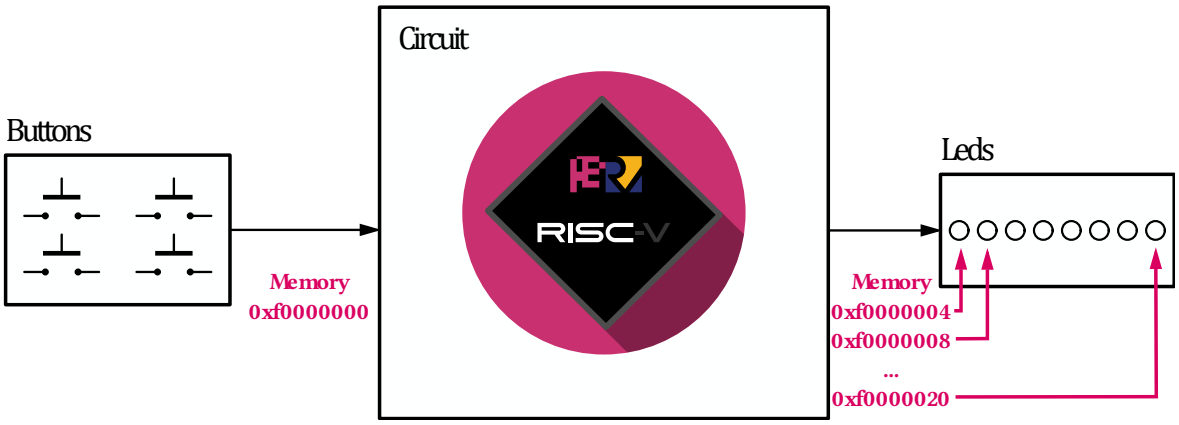

Figure 8 - Processeur RISC-V modifié

4.5.1 Donnée

Le premier laboratoire consistait à faire clignoter les leds selon les boutons appuyés:

- Aucun bouton: les 8 leds sont allumées selon le pattern `0xAA`
- Le bouton 0 est à `'1'`: les 8 leds clignotent à une fréquence raisonnable (le clignotement est visible)

Pour simplifier l'accès au bouton et aux leds, le code suivant vous était donné:

```
setup:
    # led is sw xx, offBy4(x30) with xx loaded as 0x00rrggbb
    li x30, 0xf0000004 # base address for leds
    li x31, 0xf0000000 # base address for buttons, one register

    # DO NOT MODIFY x30 (t5) and X31 (t6) !!!
    # MUST BE THE FIRST THING IN YOUR PROGRAM, BEFORE MAIN



get_btns: # return buttons value in a0
  lw a0, 0(x31)
  jr ra



set_leds: # pass a 8 bits which if bit(n) = '1', led(n) is on
  li t0, 8   # loop the leds
  mv t3, x30 # mem position
  addi t3, t3, 28 # leds display is reversed, so stock reverted

  leds_loop:
    beq x0, t0, leds_end
    andi t1, a0, 1
    # if 1, lights corresponding led
    beq t1, x0, isoff

  ison:
    li t2, 0x00ffffff
    j leds_loop_end

  isoff:
    mv t2, x0

  leds_loop_end:
    sw t2, 0(t3) # save led value
    srli a0, a0, 1  # shift right leds value
    addi t0, t0, -1 # decrement loop
    addi t3, t3, -4  # add memory pos
    j leds_loop

  leds_end:
    jr ra
```

Malheureusement, votre collègue est absent aujourd'hui et ne vous a pas laissé une copie du code déjà écrit (*pensez à apprendre Git !*). En plus, il y'avait quelques bugs:

- Aucun bouton: les leds s'allument selon le pattern **`0x2A`**
- Le bouton 0 est à **`'1'`**: seules 7 leds clignotent, et à une fréquence qui semble bien trop élevée
- Un des autres bouton est appuyé: même comportement que si le bouton 0 était appuyé

Un problème matériel peut être exclu.

4.5.2 Décompilation

Vous vous souvenez que le dernier code écrit avait été flashé sur votre board RISC-V et, vu que le chip n'avait pas été protégé contre la relecture, vous arrivez à l'extraire par accès au bus JTAG, ce qui vous a donné:

```
f0000f37
004f0f13
f0000fb7
000f8f93
00000413
040000ef
02a00463
00900663
fff48493
ff1ff06f
06400493
00800663
00000413
0140006f
0bf00413
00c0006f
02a00413
00000493
00040513
010000ef
fc5ff06f
000fa503
00008067
00800293
000f0e13
01ce0e13
02500863
00157313
00030863
010003b7
fff38393
0080006f
00000393
007e2023
00155513
fff28293
ffce0e13
fd5ff06f
00008067
```

1. Décompilez votre code
2. Identifiez et séparez le code donné du reste
3. Comprenez l'utilisation des fonctions données

4.5.3 Simulation

Pour simuler le système, sous Ripes, ouvrez l'onglet I/O:

- Double-cliquez sur Switches, sélectionnez le widget créé et configurez-le tel que:

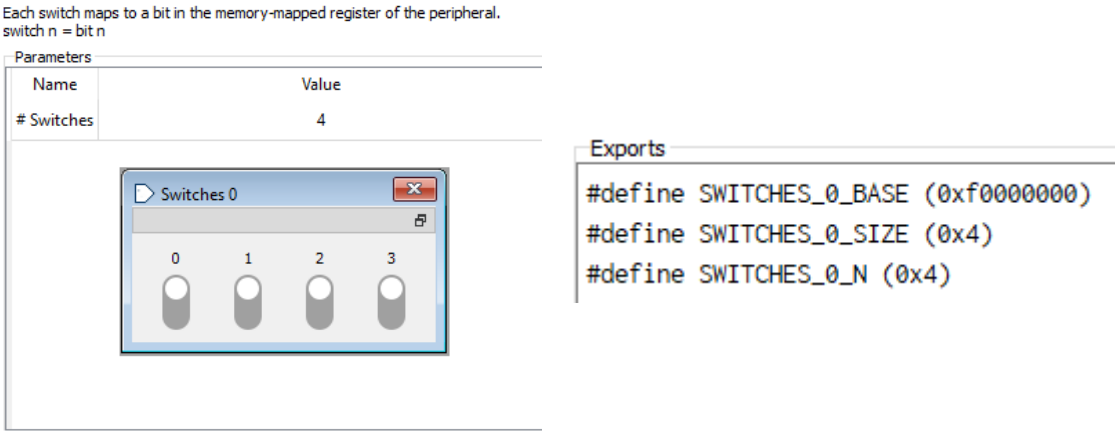

Table 2 - Ripes - configuration des interrupteurs

Contrôlez que **Exports** contienne les mêmes informations que sur l'image donnée après réglage du module.

- Double-cliquez sur LED Matrix, sélectionnez le widget créé et configurez-le tel que:

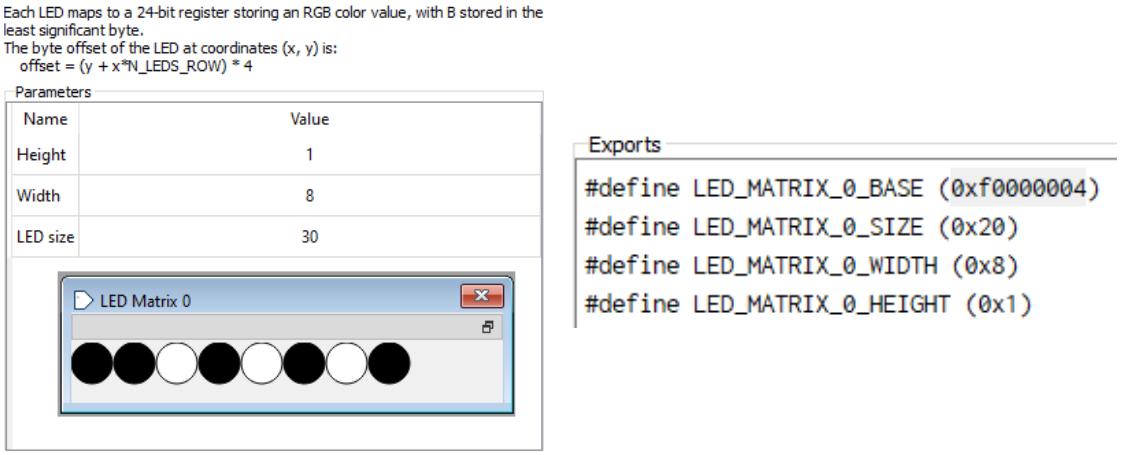

Table 3 - Ripes - configuration des leds

Contrôlez que **Exports** contienne les mêmes informations que sur l'image donnée après réglage du module.

> Ajoutez d'abord les switchs puis les leds pour obtenir la bonne addresse de base des modules.

4.5.4 Correction des erreurs

1. Chargez votre code assembleur décompilé dans **Editor**
2. Appuyez sur F8 (ou l'icône >>) et constatez que le circuit fonctionne selon les problèmes énoncés en interagissant sous **I/O** avec les interrupteurs.
3. Fixez les **4 problèmes** cités
4. Testez votre circuit sur Ripes

La clock ne pouvant être réglée de manière précise, la fréquence de clignotement des leds se fait sans calcul. Corriger le code pour obtenir environ 2 Hz.

Ce que vous venez d'accomplir s'apparente à du hacking hardware : un code est dumpé (récupéré depuis la cible), analysé, modifié, puis réinjecté dans un système pour l'adapter à son envie.

Bien sûr, il n'est pas toujours si simple de copier et réinjecter un code. Des méthodes parfois exotiques ont vu le jour (voir le Kamikaze Hack de la Xbox 360).